



- Introduzione
 - esame finale:
- Concetti fondamentali
 - qualità del software
 - Processi software
 - Metodi e strumenti
- Attività fondamentali dei processi software
 - STUDIO DI FATTIBILITÀ:
 - CODE AND FIX
 - Acquisizione, analisi e specifica dei requisiti
 - Progettazione e Sviluppo
 - Verifica e Validazione



Introduzione

si inizia di solito alle 8:45

l'anno scorso tutti sono passati tra i vari appelli

materiale didattico su elearning - corso 725

ricevicimento:

- lunedì 15:00-17:00 - L2-02-BC (A2 129)
- volendo anche su teams

esercitazioni in gruppo (2-3 persone), identici a quelli del esame, si faranno a lezione e poi avviene un confronto alla fine della lezione

libro: Ingegneria del software, Ian Sommerville, prima edizione



esame finale:

- scritto carta e penna (120 minuti)
- 3 parti:
 - esercizi su analisi e modellazione
 - esercizi di testing
 - domande di teoria

tutte e 3 le parti devono risultare sufficienti per superare l'esame
orale solo per la lode



Concetti fondamentali

cosa è software?

- è un termine che indica l'insieme dei programmi per computer e la relativa **documentazione** (anche i documenti legali)

noi terremo conto anche di tutta la documentazione prodotta durante lo sviluppo del software, come modelli di progetto, manuali utente, siti web di supporto, ecc.

definizione IEEE:

"software è l'insieme di programmi, procedure, regole e documentazione relativa al funzionamento di un sistema di elaborazione dati"

"software is eating the world" - Marc Andreessen, 2011

si sta diffondendo in tutti i settori, non solo in informatica con una capitalizzazione sempre più grande (ha superato il 1468 miliardi)

con gli LLM non è cambiato il concetto, solo la velocità in cui il software viene sviluppato. Che ci piaccia o no, LLM rimangono software e "programmare" con essi cambia solo il livello di linguaggio usato per lo sviluppo (linguaggio naturale).

Software 1.0:

vedi il problema, pianifichi un algoritmo, progetti un algoritmo e implementi l'algoritmo

Software 2.0:

prepari i dati, definisci un modello, addestri il modello e valuti il modello

Software 3.0:

prepara i dati, definisci un modello, addestri il modello, valuti il modello - iterativamente (con un transformer)

software generico:

- sw prodotto autonomamente da una organizzazione per incontrare le necessità di svariati clienti
- il produttore ha controllo sulle specifiche del sw

software custom:

- sw sviluppato da una organizzazione su commissione di uno specifico cliente
- il produttore deve attenersi alle specifiche indicate dal cliente

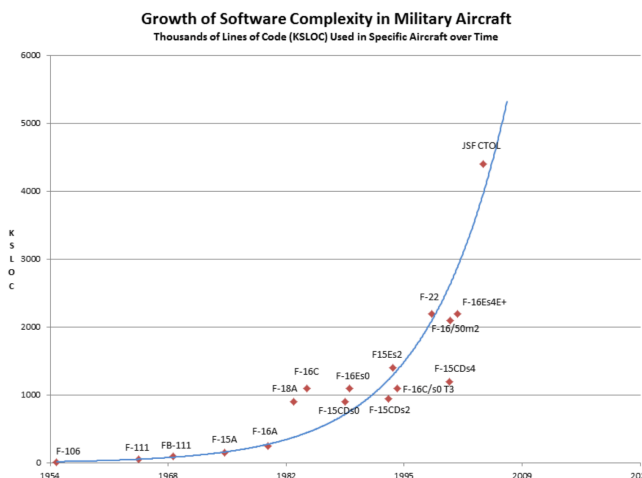


qualità del software

è importante tenere conto della complessità, perchè è fondamentale che un software sia **manutenibile**

Statisticamente, la manutenzione del codice è l'aspetto più costoso del ciclo di vita del software, quindi risolvere eventuali problemi durante la scrittura riduce drasticamente costi e tempi in fase di manutenzione.
se scopro un problema:

- durante la scrittura: costo 1
- durante la rilettura: costo 7
- durante il testing: costo 20
- dopo la consegna: costo 100



Fonte: SAVI-AVSI (2023)

Il numero di linee di codice dei sw sta crescendo esponenzialmente

Nonostante la maggiore complessità la velocità con cui il sw sta venendo sviluppato è aumentata nel tempo, evento collegato sia a strumenti per scriverlo migliori, sia allo sviluppo di approcci di ingegneria del software

il successo o l'insuccesso di un progetto sw è strettamente legato all'applicazione dell'ingegneria del sw per garantire la qualità del prodotto (anche se nel breve termine il codice potrebbe essere considerato "migliore")

un codice facilmente mantenibile rende la scrittura di nuovo codice più veloce (e ad un certo punto diventa impossibile scrivere nuovo codice senza questo processo)

Sviluppo personale

- Difficilmente il software sarà riusato in futuro da altri utenti
- Non è necessario scrivere una guida, né un documento di progetto
- I requisiti di qualità della documentazione sono meno stringenti

Sviluppo professionale

- Il software sarà usato da altre persone diverse dai propri sviluppatori
- Sviluppo in team e, quindi, probabilmente il software sarà modificato e mantenuto da altri sviluppatori
- Il software deve possedere caratteristiche di qualità



Le caratteristiche qualitative di un software professionale sono:

- **Accettabilità**
- **Fidatezza e Protezione**
- **Efficienza**
- **Mantenibilità**

L'insieme specifico dei caratteri qualitativi di un sistema software dipende dalla sua applicazione

Sviluppare sw sta diventando sempre più costoso, perchè le operazioni che un sw deve compiere sono sempre più complesse (siamo passati da giocare a pong a creare macchine a guida autonoma)

questo fenomeno venne cercato di risolvere nel 1968 alla NATO Software Engineering Conference di Garmisch, dove il problema venne definito come "**software crisis**"

La soluzione: **ingegneria del software**, che poi fece nascere la **IEEE** (Institute of Electrical and Electronics Engineers), ovvero applicare i principi dell'ingegneria ai processi di sviluppo del software

"L'Ingegneria del Software
è una disciplina
ingegneristica che
riguarda tutti gli aspetti
della produzione del software"

L'Ingegneria del Software (IS) è una disciplina ingegneristica (è l'intersezione tra ingegneria e informatica)



Processi software

Processo software: insieme di attività che porta alla creazione o all'evoluzione di un prodotto software

- Acquisizione, analisi e specifica dei requisiti
- Progettazione e Sviluppo
- Verifica e Validazione
- Evoluzione

Metodi e strumenti

Metodi:

- Approcci strutturati per sviluppare software di qualità, a costi contenuti ed entro i tempi di consegna
- Tali metodi forniscono una guida alle attività dei processi e alla relativa organizzazione

Non sono i comandamenti, ma solide linee guida sviluppate nei corsi di decenni

strumenti:

- Sistemi software usati per aiutare le attività dei processi software (ad es., analisi, modellazione, debugging, testing)

ALCUNE SFIDE PER I PROCESSI SOFTWARE:

- Diversità
- Consegna
- Fiducia
 - creare documentazione, certificazioni, ecc. per far sì che l'utente (o un altro programmatore) possa fidarsi del software
 - ci sono enti standard che certificano la qualità del software
- Scala
 - avere un sw che può essere scalato facilmente qualora il numero di utenti aumenti

PRINCIPI FONDAMENTALI:

- I metodi specifici, le tecniche e gli strumenti utilizzati dipendono dall'organizzazione che sviluppa il software, dal tipo di software e dalle persone coinvolte nel processo di sviluppo
- Non ci sono metodi di ingegneria del software universali che sono applicabili a tutti i sistemi e a tutte le aziende



- I concetti fondamentali dell'ingegneria del software sono **indipendenti dal linguaggio di programmazione** utilizzato per sviluppare il software

Il **processo** va sempre chiarito sin da subito, soprattutto quando si lavora in gruppo, altrimenti l'organizzazione e l'efficienza del gruppo collassa, si devono definire sin da subito i ruoli, le responsabilità e le attività di ogni membro del gruppo, altrimenti il progetto fallisce

FIDATEZZA E PRESTAZIONI:

il sw deve fare quello che deve fare, in maniera efficiente (e senza spreco di risorse), e in modo sicuro

REQUISITI:

- È importante capire e gestire la specifica e i requisiti del software
- Occorre sapere che cosa si aspettano clienti e utenti del sistema, e occorre gestire le loro aspettative in modo che possa essere rilasciato un sistema utile entro i costi e i tempi previsti

RIUSO:

Riutilizzare vecchio codice è spesso più rapido ed economico di scrivere nuovo codice da 0, ma bisogna stare attenti che il codice riutilizzato non abbia bug o sia effettivamente adatto al nuovo contesto.

MODELLO DI PROCESSO:

- Ciascuna organizzazione utilizza il proprio processo
- Il modello di processo software è una rappresentazione semplificata e astratta che descrive l'intero ciclo di vita del software

"come Platone e il mondo delle idee"



Attività fondamentali dei processi software

- Ciascun processo è composto da attività fondamentali
- Tutti i processi software condividono queste attività fondamentali
- Tali attività possono essere organizzate e realizzate in modi diversi in processi diversi

4 ATTIVITÀ FONDAMENTALI DEI PROCESSI SOFTWARE:

- Acquisizione, analisi e specifica dei requisiti
- Progettazione e Sviluppo
- Verifica e Validazione
- Evoluzione



STUDIO DI FATTIBILITÀ:

Stabilisce se lo sviluppo debba essere avviato, ossia:

- Se esiste un mercato per il software
- Se il software sia tecnicamente ed economicamente realistico

Definisce quali sono le alternative possibili e le scelte più ragionevoli, stimando le risorse (finanziarie e umane) necessarie per ciascuna alternativa

se in un progetto ci sono elementi già sviluppati da altri (se open source), ci permette di risparmiare tempo e denaro, se invece l'intero progetto è già stato sviluppato ad un prezzo minore di quello che potremmo proporre, spesso è il caso di abbandonare il progetto.

Fornisce un report di fattibilità con:

- Definizione del problema
- Valutazione Costi/Benefici
- Risorse finanziarie e umane
- Soluzioni alternative
- Tempi di consegna e modalità di sviluppo

Tale studio dovrebbe essere **relativamente rapido** e **poco costoso** (se sto un mese a fare uno studio sui telefoni cellulari per capire quale acquistare e nel mentre esce una nuova generazione, ho buttato il mio tempo perchè devo ricominciare da capo)



CODE AND FIX

Approccio alla produzione di software che consiste nello:

- Scrivere codice
- Aggiustare il codice per correggere errori, migliorare e/o aggiungere funzionalità

correggere errori (anche usando LLMs per fare il debugging), uno ad uno, non è l'approccio ottimale per progetti di grande dimensioni, motivo per cui si necessita di approcci più organizzati allo sviluppo del software.



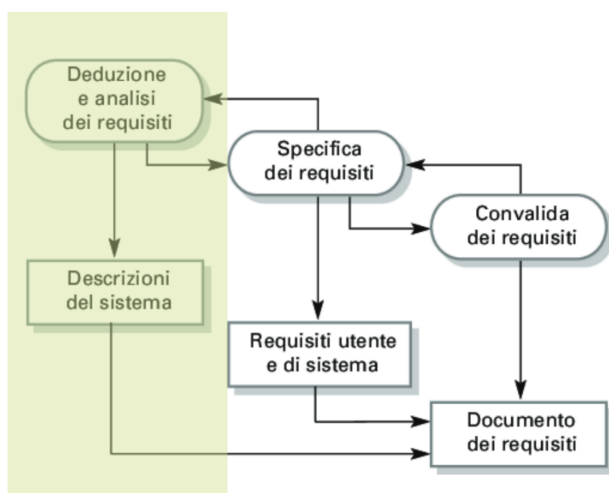
Acquisizione, analisi e specifica dei requisiti

- Attività per stabilire **cosa** il software dovrà fare
- Specificare funzionalità e qualità
- Definisce tramite l'interazione con il committente funzioni, vincoli, prestazioni, interfacce...
- Attività critica

bisogna dedurre e analizzare i requisiti del sw, non solo da parte del cliente, ma anche da parte dei **portatori di interesse** (stakeholder):

- clienti
- committenti
- enti amministrativi esterni

bisogna avere un documento formale con cui si descrive in dettaglio quali sono le funzionalità e i vincoli nel progetto (Che poi verranno convalidati da diversi enti)



La **deduzione** richiede spirito critico e può coinvolgere:

- osservazione di sistemi già esistenti
- discussione con i possibili utenti

La **specifica** traduce le informazioni dedotte in un insieme di requisiti

- Requisiti di **sistema**
- Requisiti **utente**

La **convalida** controlla che i requisiti siano

realistici, coerenti e completi

Durante questo possono essere rilevati errori nel documento dei requisiti e in tal caso il documento dei requisiti deve essere modificato e corretto

Al termine di questo processo, se il progetto rispecchia gli standard, viene approvato.

Tutte queste attività sono intrecciate, perché in ogni fase si possono notare dei difetti o errori avvenuti in una fase precedente



Progettazione e Sviluppo

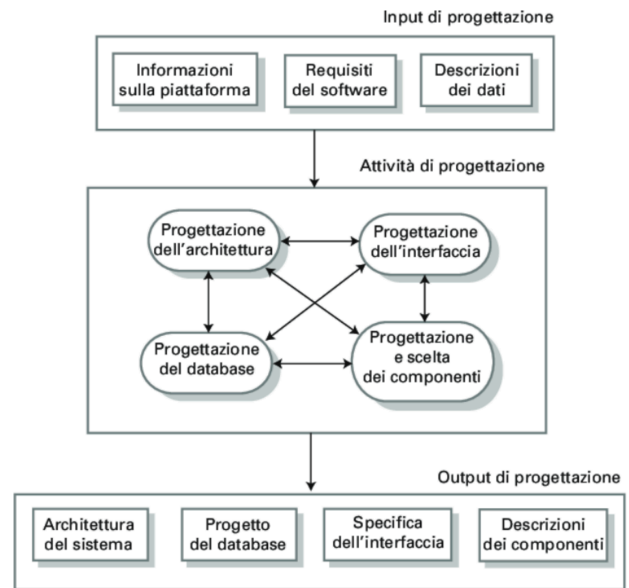
Divisa in due fasi:

- **progettazione**
- **sviluppo**

Le attività della fase di progettazione sono intrecciate e interdipendenti

I progettisti non ottengono immediatamente un risultato completo

Le nuove informazioni sul progetto influenzano le precedenti scelte progettuali



progettazione dell'architettura:

- definire classi, strutture, ecc.

progettare una interfaccia:

- non deve essere necessariamente una interfaccia grafica, ma deve essere in grado di soddisfare tutte le interazioni necessarie con l'esterno
- una volta definita una interfaccia, posso espandere il sw perchè è già suddiviso in moduli

Scelta dei componenti:

- avere componenti riutilizzabili e adattarli
- progettare componenti riutilizzabili
- bisogna mantenere i componenti flessibili, in modo che anche un altro sviluppatore possa riutilizzarli in un contesto diverso

L'output finale di queste fasi è un progetto sw (formale o informale) che descrive:

- La struttura del software che si deve implementare
- I modelli e le strutture di dati usati dal sistema
- Le interfacce tra i componenti del sistema

la parte di **sviluppo** è la meno importante in questo corso. perchè è fortemente dipendente dalle restrizioni o indicazioni di una determinata azienda, cliente, ecc...



Verifica e Validazione

nel prossimo episodio